



Aula de Git Básico

Passo a passo didático

v1.0

Iniciante

1) O que é Git? e o que é um repositório?



O que é Git?

Git é uma ferramenta que registra as mudanças feitas em arquivos ao longo do tempo. Pense nela como um histórico automático de tudo que foi alterado em um projeto.



O que é Repositório (repo)?

É o local onde esse histórico e os arquivos ficam guardados.

- > **Repositório local:** está no seu computador (uma pasta onde o Git guarda arquivos escondidos).
- > **Repositório remoto (on-line):** é uma cópia do repositório guardada na internet. Permite compartilhar e sincronizar com outras pessoas.

Repositórios mais usados



GitHub

Muito popular, fácil de usar, usado por pessoas e empresas.



GitLab

Parecido com GitHub; tem versões on-line e versões internas.



Bitbucket

Popular; integrado com outras ferramentas (ex.: Jira).

2) Conceitos-chave

Commit

Como "tirar uma foto" do estado atual dos arquivos. Cada commit tem uma mensagem explicativa.

Stage Area

Uma "bandeja" onde você coloca os arquivos que quer incluir na próxima foto (commit).

Branch

Uma "estrada paralela". Trabalhe em algo novo sem mexer na estrada principal.

Merge

Juntar o trabalho da branch de volta para a branch principal.

Remote

O repositório on-line (ex: origin é o nome comum do principal).

Clone

Criar uma cópia local de um repositório remoto.

Pull

Pegar alterações do remoto e aplicar na sua cópia local (Baixar).

Push

Enviar suas alterações locais para o remoto (Subir).

>_ 3) Principais comandos

Básicos

- > `git init` — cria um repositório Git novo na pasta atual.
- > `git clone` — copia um repositório remoto para o seu computador.
- > `git status` — mostra o estado atual (modificado, staged, etc.).
- > `git add` — coloca o arquivo na stage. `git add .` adiciona tudo.
- > `git commit -m "Msg"` — cria um commit com as mudanças staged.

Branches

- > `git branch` — lista branches locais.
- > `git branch nome` — cria uma nova.
- > `git checkout nome` — muda para a branch.
- > `git merge nome` — junta a branch indicada na atual.

Remoto

- > `git push origin main` — envia commits para o remoto.
- > `git pull` — puxa mudanças do remoto e aplica aqui.
- > `git remote add origin` — associa um remoto à pasta local.

Outros

- > `git log` — mostra o histórico de commits.
- > `.gitignore` — arquivo com lista de arquivos para o Git ignorar.



4) Passo-a-passo prático: A) Preparar o ambiente

Windows (com Git Bash), Mac ou Linux — os comandos são os mesmos.

1. Instalar Git

Ir ao site do Git e instalar. Após instalar, abrir o Terminal.

2. Configurar seu nome e e-mail (apenas da primeira vez)

Em casa digitar:

TERMINAL

```
git config --global user.name "Seu Nome"  
git config --global user.email "seu.email@exemplo.com"
```

Porque isso grava o nome e o e-mail para todos os usuários que usarem aquela máquina depois.

Aqui digitaremos:

TERMINAL

```
git config user.name "Seu Nome"
```

4) Passo-a-passo prático: B) Criar um projeto local

1. Criar pasta e entrar

```
mkdir meu-projeto  
cd meu-projeto
```

2. Inicializar o Git

Cria a pasta oculta .git.

```
git init
```

Resultado: Git informa que um repositório foi criado.

3. Criar arquivo simples

```
echo "# Meu Projeto" > README.md
```

4. Verificar o estado

```
git status
```

Resultado: Mostra README.md como não rastreado.

5. Adicionar à stage

```
git add README.md  
# ou adicionar tudo:  
git add .
```

6. Fazer primeiro commit

Agora há uma “foto” do projeto.

```
git commit -m "Primeiro commit: adiciona README"
```

4) Passo-a-passo prático: C) e D) Repositório Remoto

C) Criar no GitHub/GitLab

1. Entrar no site github.com e fazer login.
2. Clicar em "New repository".
3. Dar um nome (ex.: landing_page) e criar (sem README, pois já temos local).
4. Copiar a URL fornecida (ex.: `https://github.com/usuario/landing_page.git`).

D) Conectar e Enviar (Push)

1. Associar o remoto com nome origin:

```
git remote add origin https://github.com/usuario/landing_page.git
```

2. Renomear branch para main (se estiver como master):

```
git branch -M main
```

5) Clonar Repositório

Se alguém já tem o projeto no GitHub e você quer uma cópia:

```
git clone https://github.com/usuario/landing_page.git
```

Isso cria a pasta com todo o histórico. Depois:

```
cd meu-projeto
```

6) Fluxo de Trabalho Diário

Simples e seguro:

1 git pull
Atualizar cópia local.

2 Editar arquivos
Fazer alterações.

3 git status / git add
Ver e preparar mudanças.

4 git commit -m "..."
Salvar localmente.

5 git push
Enviar para remoto.

Sempre puxe (pull) antes de começar.

7) Branches (Ramos)

O que é? Uma linha separada de trabalho. main é a pública; crie novas para não estragar a principal.

Ciclo de Vida da Branch

1. Criar:

```
git branch nova-funcionalidade
```

2. Mudar para ela (trabalhar):

```
git checkout nova-funcionalidade  
# ou git switch nova-funcionalidade
```

3. Após commits, voltar para main:

```
git checkout main
```

4. Juntar (Merge):

```
git merge nova-funcionalidade
```

Envio e Limpeza

Enviar nova branch:

```
git push -u origin nova-funcionalidade
```

Enviar main pós-merge:

```
git push origin main
```

Deletar (se não precisa mais):

```
git branch -d nova-funcionalidade
```

⚠ 8) Conflitos

Acontece ao fazer merge/pull quando há alteração conflitante na mesma linha.

O Git marca o arquivo assim:

```
<<<<<< HEAD
Sua versão
=====
Versão que chegou
>>>>>> branch-remota
```

Resolução Simples:

1. Abra o arquivo, escolha o que manter.
2. Salve.
3. `git add e git commit.`

🚫 9) .gitignore

Arquivo onde você lista o que o Git deve **ignorar**.

Exemplo de conteúdo:

```
# temporários
.DS_Store
Thumbs.db

# node modules
node_modules/

# logs
*.log

# IDEs
.vscode/
```

Crie esse arquivo na raiz antes de commitar coisas indesejadas.

10) Comandos Úteis & 12) Erros Comuns

🔗 Recuperação / Inspeção

- > `git log --oneline`: Histórico resumido.
- > `git diff`: Ver diferenças não staged.
- > `git checkout -- arquivo`: Desfazer mudanças (restaura versão do último commit).
- > `git reset HEAD arquivo`: Tirar arquivo da stage (sem apagar mudanças).
- > `git restore --staged arquivo`: Alternativa moderna.

Atenção: alguns comandos podem perder trabalho.

🐛 Erros Comuns

Permission denied (publickey)

Problema de autenticação SSH. Use HTTPS com token ou configure chave SSH.

Failed to push some refs

Remoto tem commits novos. Solução: `git pull` (resolva conflitos se houver), depois `git push`.

Arquivos ignorados aparecendo no status?

Remover do rastreamento: `git rm --cached arquivo`.

🔗 11) Exercício Prático

Objetivo

Cada aluno cria um repositório local, envia para o GitHub, cria uma branch, faz mudanças e faz merge.

👁 Ver Resolução Passo-a-Passo

13) Glossário

- > **Commit:** Salvar ponto no tempo.
- > **Push:** Enviar para internet.
- > **Pull:** Trazer da internet.
- > **Clone:** Copiar repo da internet.
- > **Branch:** Linha paralela.
- > **Merge:** Juntar linhas.
- > **Stage:** Preparar arquivos.
- > **Remote:** Repo on-line associado.

14) Cheat-Sheet

```
# Inicializar
git init

# Clonar
git clone

# Status
git status

# Adicionar
git add .

# Commitar
git commit -m "msg"

# Histórico
git log --oneline

# Enviar / Trazer
git push origin main
git pull
```